Greedy Approximation for Type Error Correction: A Diagnostic Study on HM-Style Constraint Systems

Kevin Chen
COMP4600/8460 Advanced Algorithms

Semester 1, 2026

Public repository: `https://github.com/Provia/hm-mcs-approx`

Abstract

Type error localization in Hindley–Milner systems can be formulated as a minimum-weight Minimum Correction Subset (MWMCS) problem over unsatisfiable type-equality constraints. We empirically study a lazy MUS-guided greedy hitting-set approximation against an exact Z3 Optimize baseline on 300 synthetic instances spanning three structural families: single planted conflicts, overlapping conflicts on a shared type variable, and constructor-level function–list mismatches. Across all instances, the greedy algorithm matches the exact optimum (approximation ratio uniformly 1.0), and brute-force enumeration independently validates the Z3 baseline on small instances. We do not interpret this as evidence that greedy is generally optimal; rather, we identify the structural reasons each generator family fails to exercise the standard worst-case behaviour of greedy hitting set. Runtime analysis shows that lazy greedy overhead is driven by the number of MUSes discovered rather than the total constraint count, producing a stable 2–3× slowdown on the dataset with overlapping conflicts. The main contribution is a diagnostic characterisation of when synthetic HM-style generators do and do not produce approximation gaps, which motivates future work on adversarial generators inspired by Set Cover hardness templates.

1 Motivation

1.1 Type Errors in Hindley–Milner Systems

Hindley–Milner type inference operates by generating a set of type-equality constraints from a program's syntax and solving them via Robinson unification [1]. When the program is ill-typed, this constraint set is unsatisfiable, and the compiler must report an error location. Standard implementations report the first unification failure encountered during constraint solving, a location that is structurally determined by traversal order rather than by any minimality criterion. As a consequence, the reported location may not correspond to the actual source of the type error, making error messages brittle and difficult to act on [6].

1.2 The Algorithmic Significance

A principled alternative is to identify a minimum-weight subset of constraints whose removal restores satisfiability. This is a combinatorial optimization problem. Via the MCS–MUS hitting-set duality established by Marques-Silva et al. [5], this problem is directly related to finding a minimum-weight hitting set over the family of Minimal Unsatisfiable Subsets of the constraint system. Existing work addresses related localization problems using exact MaxSMT solvers [4], [6] or MCS enumeration [3], both of which provide principled answers but can become expensive as instance size grows. The literature reviewed here does not provide polynomial-time approximation guarantees or approximation-hardness results for this problem in the HM setting.

1.3 Why Approximation Theory Matters Here

Exact methods can be impractical for large programs, while heuristics without quality guarantees offer no formal assurance on solution quality. A polynomial-time approximation algorithm with a provable ratio would fill this gap by trading optimality for scalability in a controlled, quantifiable way. This paper takes a first empirical step: we implement a greedy approximation algorithm, evaluate its approximation ratio against an exact baseline on synthetic HM-style instances, and diagnose which structural features of those instances drive its behaviour.

2 Formulation

2.1 Constraint Systems in Hindley–Milner Inference

Hindley–Milner type inference reduces to solving a finite system of type-equality constraints generated by a syntax-directed traversal of the program. Formally, let $C = \{c_1, \ldots, c_m\}$ be a set of constraints of the form $\tau_i = \tau_j$, where each τ_k is a type expression over a set of type variables. Each constraint c_i carries a positive weight $w_i \in \mathbb{R}_{>0}$ reflecting the confidence assigned to that typing judgment [6]. The system C is *satisfiable* if there exists a substitution σ such that $\sigma(\tau_i) = \sigma(\tau_j)$ for all $c_i \in C$; otherwise it is *unsatisfiable*.

2.2 Minimal Correction Subsets and the MWMCS Problem

When C is unsatisfiable, a *Minimal Correction Subset* is a subset $M \subseteq C$ such that $C \setminus M$ is satisfiable, and no proper subset of M has this property [5]. Each element of an MCS is a candidate error location.

Worked example. Consider the program `let f x = x + 1 in f True`. HM inference generates type-equality constraints

$$c_0 : \ \alpha = \texttt{Int}, \qquad c_1 : \ \alpha = \texttt{Bool},$$

together with satisfiable constraints binding intermediate variables in the function definition. The set $\{c_0, c_1\}$ is the unique MUS, since unification of `Int` and `Bool` fails. With unit weights, either $\{c_0\}$ or $\{c_1\}$ is a minimum-weight correction subset of total weight 1.0.

The *Minimum-Weight MCS* problem asks:

> Given an unsatisfiable weighted constraint system (C, w), find a correction subset $M^* \subseteq C$ minimising $\sum_{c \in M^*} w_c$.

By the hitting-set duality of [5], every correction subset is a hitting set of the family of Minimal Unsatisfiable Subsets of C, and every hitting set of all MUSes is a correction subset. MWMCS is therefore directly related to Minimum-Weight Hitting Set on the MUS hypergraph. The general hitting-set problem is NP-hard and is inapproximable within $(1 - \epsilon) \ln |C|$ for any $\epsilon > 0$ unless P = NP [2].

2.3 Research Question and Scope

This paper investigates the following question empirically:

> *On HM-shaped synthetic constraint instances, does a greedy hitting-set approximation achieve a ratio strictly greater than* 1 *relative to the exact MWMCS solution, and if not, what structural properties of the instances explain the absence of a gap?*

We restrict scope to three synthetic generator families, described in Section 3, spanning a single-conflict structure Dataset A, overlapping conflicts on a shared type variable Dataset B, and HM-shaped constructor mismatches Dataset C. For each family, we evaluate $|C| \in \{10, 20, 50, 100, 200\}$ with 20 instances per cell. We use an exact MaxSMT-style baseline implemented with Z3 as the reference optimum and do not claim a theoretical approximation bound for the lazy implementation.

3 Methodology

3.1 Synthetic Instance Generators

We evaluate on three families of synthetic HM-style constraint systems, each designed to isolate a distinct structural property. We use synthetic generators to keep the source, conflict structure, and weighting scheme fully reproducible.

Dataset A: Single Planted Conflict. Each instance contains exactly one conflict pair: two constraints binding the same fresh type variable α to incompatible base types, $\alpha = \texttt{Int}$ and $\alpha = \texttt{Bool}$, both with weight 1.0. The remaining $|C| - 2$ constraints are satisfiable fillers over independent fresh variables, including base, list, and function types. This yields a unique MUS of size 2 and two symmetric minimum-weight correction subsets, each of weight 1.0.

Dataset B: Overlapping Conflicts. Each instance plants $k = 4$ constraints binding a shared variable x to mutually incompatible types $\{\texttt{Int}, \texttt{Bool}, \texttt{Char}, \texttt{String}\}$ with weights $1.0, 2.0, 3.0, 4.0$ respectively. Every pair of these constraints forms a MUS, yielding $\binom{k}{2} = 6$ pairwise MUSes, all involving the same shared variable. The satisfiable fillers are appended as in Dataset A. This family tests whether MUS overlap alone is enough to produce an approximation gap.

Dataset C: HM-Shaped Constructor Mismatch. Each instance plants a conflict between a function-type shape and a list-type shape on the same variable f: the constraints $f = [\alpha] \to \beta$, $\alpha = \texttt{Int}$, $\beta = \texttt{Bool}$, and $f = [\texttt{Char}]$ are inconsistent because f cannot simultaneously be a function type and a list type. Constraint weights are derived from type-expression depth:

$$w_i = 1 + \max(\text{depth}(\tau_i^L), \text{depth}(\tau_i^R)).$$

Fillers are randomly generated type expressions up to depth 3.

Worked example. Consider `let x = 5 in (x ++ "hello", x && True)`. The variable `x` receives a fresh type variable τ, and its three occurrences impose

$$c_0 : \ \tau = \texttt{Int}, \qquad c_1 : \ \tau = [\texttt{Char}], \qquad c_2 : \ \tau = \texttt{Bool}.$$

Each pair $\{c_i, c_j\}$ is a MUS, yielding three pairwise MUSes that overlap on the shared variable τ. This matches the planted structure of Dataset B and supports the interpretation that B contains genuine MUS overlap, even though that overlap is too regular to produce an approximation gap.

3.2 Greedy Approximation Algorithm

We implement a lazy MUS-guided greedy hitting-set algorithm. At each iteration, the algorithm invokes a MUS finder on the current residual constraint set to obtain one MUS M, adds M to the set of discovered MUSes, and recomputes a weighted greedy hitting set over all MUSes discovered so far. At each greedy hitting-set step, it selects the constraint

$$c^* = \arg\min_c \frac{w_c}{|\{M' : c \in M'\}|},$$

where the denominator counts currently unhit discovered MUSes containing c. The selected hitting set is removed from the original constraint set, and the process repeats until the residual constraint set is satisfiable.

MUS extraction uses an unsatisfiable-core procedure followed by deletion-based shrinking: constraints are tentatively removed one at a time and kept removed if the subset remains unsatisfiable. Satisfiability at each step is checked by Robinson unification over the type-equality constraint system. The full-information weighted hitting-set greedy algorithm has the standard H_d approximation bound, where d is the maximum set size. Our implementation uses this greedy rule lazily on discovered MUSes and is therefore evaluated empirically against the exact baseline.

3.3 Exact Baseline

We use Z3's optimisation interface as the exact baseline. Each constraint c_i is guarded by a Boolean variable $keep_i$. If $keep_i$ is true, the corresponding type equality must hold. The objective minimises the total weight of dropped constraints, namely constraints for which $keep_i$ is false. This returns an exact minimum-weight correction subset for the encoded instance. A timeout of 30,000 ms per instance is applied; all 300 instances completed within budget.

3.4 Brute-Force Sanity Check

To validate the Z3 baseline, we run exhaustive brute-force search on small instances with $|C| = 10$. This sanity check covers 15 instances, from 5 seeds across the 3 datasets. The brute-force solver enumerates all $2^{|C|}$ candidate correction subsets and returns the minimum-weight one. Z3 and brute force agree on optimal weight for all 15 instances.

3.5 Experimental Grid

The full experiment spans 3 datasets, 5 sizes $|C| \in \{10, 20, 50, 100, 200\}$, and 20 seeds, giving 300 instances. For each instance we record greedy weight, exact weight, approximation ratio, greedy and exact wall-clock runtime, number of greedy iterations, number of MUSes discovered, MUS sizes, overlap measure, and weight statistics. Code and results are available at `https://github.com/Provia/hm-mcs-approx`.

4 Results

4.1 Approximation Ratio

Across all 300 instances—3 datasets × 5 sizes × 20 seeds—the greedy algorithm matched the exact Z3 solution on every instance. The approximation ratio $\rho = w_{\text{greedy}}/w_{\text{exact}}$ was exactly 1.00 in all cases, with zero variance. This result holds uniformly across $|C| \in \{10, 20, 50, 100, 200\}$ and is not an artefact of solver failure: the exact success rate was 1.0 throughout, and brute-force enumeration independently confirmed Z3's optima on all 15 small instances ($|C| = 10$).

A ratio of 1.00 on every instance does not imply greedy is generally optimal. Rather, it reflects the structural simplicity of the three generator families, as we analyse below.

4.2 Structural Explanation by Dataset

Dataset A. Each instance contains a single MUS of size 2 with equal-weight constraints. Greedy discovers this MUS in one iteration, selects either element, and halts. Because the MUS has only two elements of equal weight, any singleton correction subset is optimal; greedy cannot diverge from exact. Mean greedy iterations: 1.00; mean MUSes discovered: 1.00.

Figure 1: Runtime scaling of greedy and exact baselines

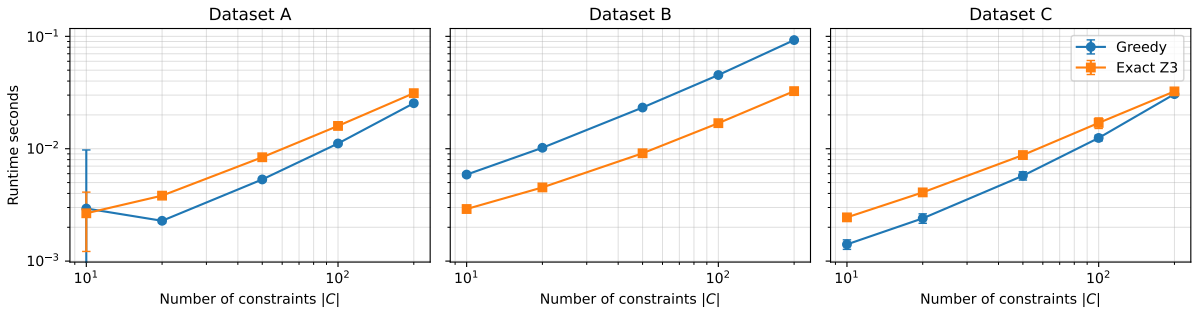

Figure 1: Runtime scaling of greedy and exact Z3 baselines on log–log axes. Dataset B is consistently slower for greedy due to its 5-iteration MUS discovery structure; Datasets A and C converge toward exact runtime at larger $|C|$.

Dataset B. Each instance plants 4 constraints on a shared variable x with weights 1.0, 2.0, 3.0, 4.0, producing $\binom{4}{2} = 6$ pairwise MUSes in the planted core. The lazy greedy run discovers 5 MUSes before the residual instance becomes satisfiable. In this generated clique-like structure, the greedy choices coincide with the exact optimum. Mean greedy iterations: 5.00; mean MUSes discovered: 5.00, constant across sizes because the planted conflict structure does not grow with $|C|$. Greedy runtime (35.45 ms mean) exceeds exact runtime (13.19 ms mean) on this dataset because greedy performs 5 sequential MUS-extraction passes while Z3 solves the optimisation encoding in a single call.

Dataset C. Each instance plants a constructor mismatch: f is forced to be both a function type and a list type. The resulting MUS contains exactly the two conflicting constraints (`C_shape_function` and `C_conflict_list`); the other planted constraints ($\alpha = \texttt{Int}$, $\beta = \texttt{Bool}$) are consistent with either resolution and do not participate in a minimal conflict. Greedy discovers this MUS in one iteration and removes one of its elements. Mean greedy iterations: 1.00; mean MUSes discovered: 1.00.

4.3 Runtime Scaling

Greedy runtime grows with $|C|$ across all datasets, driven by the cost of unification checks during MUS extraction (Figure 1). On Dataset A, mean greedy runtime increases from 2.94 ms at $|C| = 10$ to 25.41 ms at $|C| = 200$. Dataset C shows a similar profile (1.41 ms to 30.61 ms). Dataset B is slower due to its 5-iteration structure, rising from 5.89 ms to 92.64 ms. Exact Z3 runtime is comparatively stable across sizes and datasets (12–13 ms mean), suggesting that, on these small synthetic instances, the fixed overhead of the Z3 optimisation call dominates the additional filler constraints.

Figure 2 shows that greedy overhead relative to exact is explained by MUS discovery count. Datasets A and C discover exactly 1 MUS per instance regardless of $|C|$, keeping the relative runtime at or below 1.0 for larger sizes. Dataset B discovers exactly 5 MUSes per instance, one per greedy iteration, and its relative runtime grows from about 2.0× at $|C| = 10$ to about 2.8× at $|C| = 200$.

4.4 Interpretation: Generator Simplicity, Not Algorithm Optimality

The absence of an approximation gap is a property of the generators, not a general property of the greedy algorithm. All three datasets produce MUS hypergraphs that are structurally easy for weighted greedy: Dataset A has a unique two-element MUS with equal weights; Dataset B

Figure 2: Lazy greedy overhead is driven by MUS discovery

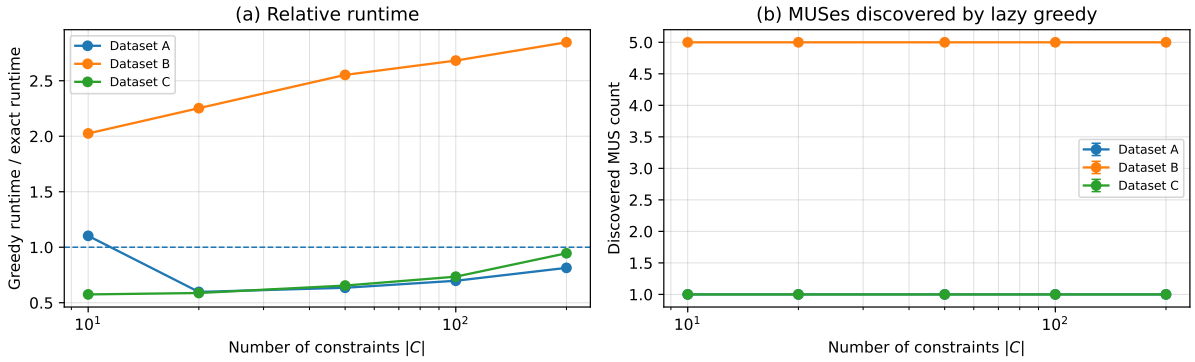

Figure 2: (a) Greedy runtime relative to exact Z3. Dataset B consistently exceeds 2× owing to 5 sequential MUS-extraction passes. (b) MUSes discovered by lazy greedy per instance. MUS count is constant across all sizes for each dataset, confirming that conflict complexity does not scale with $|C|$ in the current generators.

has a regular clique-like structure on one variable; Dataset C reduces to a two-element MUS after constraint propagation.

Adversarial instances for Set-Cover-style greedy require MUSes with disjoint support and carefully constructed weights so that the greedy choice at each step is locally cheap but globally suboptimal. None of the three generator families produce such structure, because the planted conflicts are confined to one or two variables with straightforward weight assignments. Constructing generators that force a gap—for example, by planting multiple disjoint MUSes with cross-cutting weight gradients inspired by the Set Cover hardness template of [2]—is a natural direction for future work.

5 Conclusion and Discussion

This paper studied a lazy MUS-guided greedy hitting-set algorithm for the Minimum-Weight Minimum Correction Subset (MWMCS) problem on synthetic HM-style type-equality constraint systems. Across 300 instances spanning three generator families and five sizes, the greedy algorithm matched the exact Z3 optimum on every instance, with approximation ratio uniformly equal to one. The brute-force baseline independently confirmed the Z3 optima on small instances. Runtime analysis showed that lazy greedy overhead is driven by the number of MUSes discovered rather than by the total constraint count: Datasets A and C, each with one MUS per instance, ran at or below exact runtime, while Dataset B's five-iteration loop produced a consistent 2–3× slowdown that was stable across instance sizes.

Interpretation. The absence of an approximation gap describes the tested instance space, not the algorithm. A greedy hitting-set algorithm carries a worst-case bound of $1 + \ln d$, where d is the maximum MUS size, and this bound remains applicable to instances outside the tested space. The empirical result here is that the three synthetic generators produce structurally simple MUS hypergraphs—a single small MUS in Datasets A and C, and a regular clique on a single shared variable in Dataset B—on which weighted greedy happens to be optimal. The experiment therefore characterises the generators as much as it characterises the algorithm.

Limitations. Three limitations should be stated explicitly. First, the synthetic generators are HM-shaped rather than HM-derived: they encode plausible type-equality structure but were not produced by a real Haskell or OCaml compiler. Second, only one greedy variant was evaluated;

alternative strategies, such as weighted greedy with full MUS enumeration or LP-rounding, were not compared. Third, the instance sizes ($|C| \leq 200$) are small relative to real-world programs, where MUS hypergraph structure is likely richer.

Future work. The most direct extension is to construct adversarial generators that exercise the worst-case greedy behaviour, for example by planting multiple disjoint MUSes with cross-cutting weight gradients in the style of the Set Cover hardness template of Feige [2]. A complementary direction, returning to the theoretical question framed in our preliminary proposal phase, is to establish whether logarithmic inapproximability transfers from Set Cover to MWMCS on HM-derived instance classes, or whether the AST-derived structure of HM constraints admits strictly better bounds. Empirical evaluation on real type-error corpora, contingent on access to a reusable benchmark, remains an open opportunity.

References

[1] L. Damas and R. Milner, "Principal type-schemes for functional programs," in *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '82)*, ACM, 1982, pp. 207–212. DOI: 10.1145/582153.582176

[2] U. Feige, "A threshold of $\ln n$ for approximating set cover," *Journal of the ACM*, vol. 45, no. 4, pp. 634–652, 1998. DOI: 10.1145/285055.285059

[3] S. Fu, T. Dwyer, P. J. Stuckey, and J. Grundy, "Goanna: Resolving Haskell type errors with minimal correction subsets," in *Proceedings of the ACM on Programming Languages (ICFP '24)*, arXiv:2405.12697, ACM, 2024.

[4] M. Kopinsky, B. Pientka, and X. Si, *Modernizing SMT-based type error localization*, arXiv preprint arXiv:2408.09034, 2024. [Online]. Available: https://arxiv.org/abs/2408.09034

[5] J. Marques-Silva, F. Heras, M. Janota, A. Previti, and A. Belov, "On computing minimal correction subsets," in *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI '13)*, AAAI Press, 2013, pp. 615–622.

[6] Z. Pavlinovic, T. King, and T. Wies, "Finding minimum type error sources," in *Proceedings of the 2014 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '14)*, ACM, 2014, pp. 525–542. DOI: 10.1145/2660193.2660230